JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class PrintStream

java.lang.Object
    java.io.OutputStream
        java.io.FilterOutputStream
            java.io.PrintStream

All Implemented Interfaces:

`Closeable`, `Flushable`, `Appendable`, `AutoCloseable`

Direct Known Subclasses:

`LogStream`

```
public class PrintStream
extends FilterOutputStream
implements Appendable, Closeable
```

A `PrintStream` adds functionality to another output stream, namely the ability to print representations of various data values conveniently. Two other features are provided as well. Unlike other output streams, a `PrintStream` never throws an `IOException`; instead, exceptional situations merely set an internal flag that can be tested via the `checkError` method. Optionally, a `PrintStream` can be created so as to flush automatically; this means that the `flush` method of the underlying output stream is automatically invoked after a byte array is written, one of the `println` methods is invoked, or a newline character or byte (`'\n'`) is written.

All characters printed by a `PrintStream` are converted into bytes using the given encoding or charset, or the default charset if not specified. The `PrintWriter` class should be used in situations that require writing characters rather than bytes.

This class always replaces malformed and unmappable character sequences with the charset's default replacement string. The CharsetEncoder class should be used when more control over the encoding process is required.

Since:

1.0

See Also:

`Charset.defaultCharset()`

Field Summary

Fields declared in class FilterOutputStream

`out`

Constructor Summary

Constructors

Constructor	Description
`PrintStream(File file)`	Creates a new print stream, without automatic line flushing, with the specified file.
`PrintStream(File file, String csn)`	Creates a new print stream, without automatic line flushing, with the specified file and charset.
`PrintStream(File file, Charset charset)`	Creates a new print stream, without automatic line flushing, with the specified file and charset.
`PrintStream(OutputStream out)`	Creates a new print stream, without automatic line flushing, with the specified OutputStream.
`PrintStream(OutputStream out, boolean autoFlush)`	Creates a new print stream, with the specified OutputStream and line flushing.
`PrintStream(OutputStream out, boolean autoFlush, String encoding)`	Creates a new print stream, with the specified OutputStream, line flushing, and character encoding.
`PrintStream(OutputStream out, boolean autoFlush, Charset charset)`	Creates a new print stream, with the specified OutputStream, line flushing and charset.
`PrintStream(String fileName)`	Creates a new print stream, without automatic line flushing, with the specified file name.
`PrintStream(String fileName, String csn)`	Creates a new print stream, without automatic line flushing, with the specified file name and charset.
`PrintStream(String fileName, Charset charset)`	Creates a new print stream, without automatic line flushing, with the specified file name and charset.

Method Summary

All Methods	Instance Methods	

Modifier and Type	Method	Description
PrintStream	**append**(char c)	Appends the specified character to this output stream.
PrintStream	**append**(CharSequence csq)	Appends the specified character sequence to this output stream.
PrintStream	**append**(CharSequence csq, int start, int end)	Appends a subsequence of the specified character sequence to this output stream.
Charset	**charset**()	Returns the charset used in this PrintStream instance.
boolean	**checkError**()	Flushes the stream if it's not closed and checks its error state.
protected void	**clearError**()	Clears the error state of this stream.
void	**close**()	Closes the stream.
void	**flush**()	Flushes the stream.
PrintStream	**format**(String format, Object... args)	Writes a formatted string to this output stream using the specified format string and arguments.
PrintStream	**format**(Locale l, String format, Object... args)	Writes a formatted string to this output stream using the specified format string and arguments.
void	**print**(boolean b)	Prints a boolean value.

void	**print**(char c)	Prints a character.
void	**print**(char[] s)	Prints an array of characters.
void	**print**(double d)	Prints a double-precision floating-point number.
void	**print**(float f)	Prints a floating-point number.
void	**print**(int i)	Prints an integer.
void	**print**(long l)	Prints a long integer.
void	**print**(Object obj)	Prints an object.
void	**print**(String s)	Prints a string.
PrintStream	**printf**(String format, Object... args)	A convenience method to write a formatted string to this output stream using the specified format string and arguments.
PrintStream	**printf**(Locale l, String format, Object... args)	A convenience method to write a formatted string to this output stream using the specified format string and arguments.
void	**println**()	Terminates the current line by writing the line separator string.
void	**println**(boolean x)	Prints a boolean and then terminates the line.
void	**println**(char x)	Prints a character and then terminates the line.
void	**println**(char[] x)	Prints an array of characters and then terminates the line.
void	**println**(double x)	

		Prints a double and then terminates the line.
void	**println**(float x)	Prints a float and then terminates the line.
void	**println**(int x)	Prints an integer and then terminates the line.
void	**println**(long x)	Prints a long and then terminates the line.
void	**println**(Object x)	Prints an Object and then terminates the line.
void	**println**(String x)	Prints a String and then terminates the line.
protected void	**setError**()	Sets the error state of the stream to `true`.
void	**write**(byte[] buf)	Writes all bytes from the specified byte array to this stream.
void	**write**(byte[] buf, int off, int len)	Writes `len` bytes from the specified byte array starting at offset `off` to this stream.
void	**write**(int b)	Writes the specified byte to this stream.
void	**writeBytes**(byte[] buf)	Writes all bytes from the specified byte array to this stream.

Methods declared in class OutputStream

`nullOutputStream`

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Constructor Details

PrintStream

```
public PrintStream(OutputStream out)
```

Creates a new print stream, without automatic line flushing, with the specified OutputStream. Characters written to the stream are converted to bytes using the default charset, or where `out` is a `PrintStream`, the charset used by the print stream.

Parameters:

`out` - The output stream to which values and objects will be printed

See Also:

`PrintWriter(java.io.OutputStream)`, `Charset.defaultCharset()`

PrintStream

```
public PrintStream(OutputStream out,
                   boolean autoFlush)
```

Creates a new print stream, with the specified OutputStream and line flushing. Characters written to the stream are converted to bytes using the default charset, or where `out` is a `PrintStream`, the charset used by the print stream.

Parameters:

`out` - The output stream to which values and objects will be printed

`autoFlush` - Whether the output buffer will be flushed whenever a byte array is written, one of the `println` methods is invoked, or a newline character or byte (`'\n'`) is written

See Also:

`PrintWriter(java.io.OutputStream, boolean)`, `Charset.defaultCharset()`

PrintStream

```
public PrintStream(OutputStream out,
                   boolean autoFlush,
                   String encoding)
            throws UnsupportedEncodingException
```

Creates a new print stream, with the specified OutputStream, line flushing, and character encoding.

Parameters:

`out` - The output stream to which values and objects will be printed

`autoFlush` - Whether the output buffer will be flushed whenever a byte array is written, one of the `println` methods is invoked, or a newline character or byte (`'\n'`) is written

`encoding` - The name of a supported character encoding

Throws:

`UnsupportedEncodingException` - If the named encoding is not supported

Since:

1.4

PrintStream

```
public PrintStream(OutputStream out,
                   boolean autoFlush,
                   Charset charset)
```

Creates a new print stream, with the specified OutputStream, line flushing and
charset. This convenience constructor creates the necessary intermediate `OutputStreamWriter`, which will encode characters using the provided charset.

Parameters:

`out` - The output stream to which values and objects will be printed

`autoFlush` - Whether the output buffer will be flushed whenever a byte array is written, one of the `println` methods is invoked, or a newline character or byte (`'\n'`) is written

`charset` - A charset

Since:

10

PrintStream

```
public PrintStream(String fileName)
            throws FileNotFoundException
```

Creates a new print stream, without automatic line flushing, with the specified file name. This convenience constructor creates the necessary intermediate `OutputStreamWriter`, which will encode characters using the default charset for this instance of the Java virtual machine.

Parameters:

`fileName` - The name of the file to use as the destination of this print stream. If the file exists, then it will be truncated to zero size; otherwise, a new file will be created. The output will be written to the file and is buffered.

Throws:

`FileNotFoundException` - If the given file object does not denote an existing, writable regular file and a new regular file of that name cannot be created, or if some other error occurs while opening or creating the file

Since:

1.5

See Also:

`Charset.defaultCharset()`

PrintStream

```
public PrintStream(String fileName,
                   String csn)
            throws FileNotFoundException,
                   UnsupportedEncodingException
```

Creates a new print stream, without automatic line flushing, with the specified file name and charset. This convenience constructor creates the necessary intermediate `OutputStreamWriter`, which will encode characters using the provided charset.

Parameters:

`fileName` - The name of the file to use as the destination of this print stream. If the file exists, then it will be truncated to zero size; otherwise, a new file will be created. The output will be written to the file and is buffered.

`csn` - The name of a supported charset

Throws:

`FileNotFoundException` - If the given file object does not denote an existing, writable regular file and a new regular file of that name cannot be created, or if some other error occurs while opening or creating the file

`UnsupportedEncodingException` - If the named charset is not supported

Since:

1.5

PrintStream

```
public PrintStream(String fileName,
                   Charset charset)
            throws IOException
```

Creates a new print stream, without automatic line flushing, with the specified file name and charset. This convenience constructor creates the necessary intermediate `OutputStreamWriter`, which will encode characters using the provided charset.

Parameters:

`fileName` - The name of the file to use as the destination of this print stream. If the file exists, then it will be truncated to zero size; otherwise, a new file will be created. The output will be written to the file and is buffered.

`charset` - A charset

Throws:

`IOException` - if an I/O error occurs while opening or creating the file

Since:

10

PrintStream

```
public PrintStream(File file)
            throws FileNotFoundException
```

Creates a new print stream, without automatic line flushing, with the specified file. This convenience constructor creates the necessary intermediate `OutputStreamWriter`, which will encode characters using the default charset for this instance of the Java virtual machine.

Parameters:

`file` - The file to use as the destination of this print stream. If the file exists, then it will be truncated to zero size; otherwise, a new file will be created. The output will be written to the file and is buffered.

Throws:

`FileNotFoundException` - If the given file object does not denote an existing, writable regular file and a new regular file of that name cannot be created, or if some other error occurs while opening or creating the file

Since:

1.5

See Also:

`Charset.defaultCharset()`

PrintStream

```
public PrintStream(File file,
                   String csn)
            throws FileNotFoundException,
                   UnsupportedEncodingException
```

Creates a new print stream, without automatic line flushing, with the specified file and charset. This convenience constructor creates the necessary intermediate `OutputStreamWriter`, which will encode characters using the provided charset.

Parameters:

`file` - The file to use as the destination of this print stream. If the file exists, then it will be truncated to zero size; otherwise, a new file will be created. The output will be written to the file and is buffered.

`csn` - The name of a supported charset

Throws:

`FileNotFoundException` - If the given file object does not denote an existing, writable regular file and a new regular file of that name cannot be created, or if some other error occurs while opening or creating the file

`UnsupportedEncodingException` - If the named charset is not supported

Since:

1.5

PrintStream

```
public PrintStream(File file,
                   Charset charset)
            throws IOException
```

Creates a new print stream, without automatic line flushing, with the specified file and charset. This convenience constructor creates the necessary intermediate `OutputStreamWriter`, which will encode characters using the provided charset.

Parameters:

`file` - The file to use as the destination of this print stream. If the file exists, then it will be truncated to zero size; otherwise, a new file will be created. The output will be written to the file and is buffered.

`charset` - A charset

Throws:

`IOException` - if an I/O error occurs while opening or creating the file

Since:

10

Method Details

flush

`public void flush()`

Flushes the stream. This is done by writing any buffered output bytes to the underlying output stream and then flushing that stream.

Specified by:

`flush` in interface `Flushable`

Overrides:

`flush` in class `FilterOutputStream`

See Also:

`OutputStream.flush()`

close

`public void close()`

Closes the stream. This is done by flushing the stream and then closing the underlying output stream.

Specified by:

`close` in interface `AutoCloseable`

Specified by:

`close` in interface `Closeable`

Overrides:

`close` in class `FilterOutputStream`

See Also:

`OutputStream.close()`

checkError

```
public boolean checkError()
```

Flushes the stream if it's not closed and checks its error state.

Returns:

`true` if and only if this stream has encountered an `IOException`, or the `setError` method has been invoked

setError

```
protected void setError()
```

Sets the error state of the stream to `true`.

This method will cause subsequent invocations of `checkError()` to return `true` until `clearError()` is invoked.

Since:

1.1

clearError

```
protected void clearError()
```

Clears the error state of this stream.

This method will cause subsequent invocations of `checkError()` to return `false` until another write operation fails and invokes `setError()`.

Since:

1.6

write

```
public void write(int b)
```

Writes the specified byte to this stream. If the byte is a newline and automatic flushing is enabled then the `flush` method will be invoked on the underlying output stream.

Note that the byte is written as given; to write a character that will be translated according to the default charset, use the `print(char)` or `println(char)` methods.

Overrides:

`write` in class `FilterOutputStream`

Parameters:

`b` - The byte to be written

See Also:

`print(char)`, `println(char)`

write

```
public void write(byte[] buf,
                  int off,
                  int len)
```

Writes `len` bytes from the specified byte array starting at offset `off` to this stream. If automatic flushing is enabled then the `flush` method will be invoked on the underlying output stream.

Note that the bytes will be written as given; to write characters that will be translated according to the default charset, use the `print(char)` or `println(char)` methods.

Overrides:

`write` in class `FilterOutputStream`

Parameters:

`buf` - A byte array

`off` - Offset from which to start taking bytes

`len` - Number of bytes to write

Throws:

`IndexOutOfBoundsException` - If `off` is negative, `len` is negative, or `len` is greater than `b.length - off`

See Also:

`FilterOutputStream.write(int)`

write

```
public void write(byte[] buf)
           throws IOException
```

Writes all bytes from the specified byte array to this stream. If automatic flushing is enabled then the `flush` method will be invoked on the underlying output stream.

Note that the bytes will be written as given; to write characters that will be translated according to the default charset, use the `print(char[])` or `println(char[])` methods.

Overrides:

`write` in class `FilterOutputStream`

API Note:

Although declared to throw `IOException`, this method never actually does so. Instead, like other methods that this class overrides, it sets an internal flag which may be tested via the `checkError()` method. To write an array of bytes without having to write a `catch` block for the `IOException`, use either `writeBytes(buf)` or `write(buf, 0, buf.length)`.

Implementation Requirements:

This method is equivalent to `this.write(buf, 0, buf.length)`.

Parameters:

`buf` - A byte array

Throws:

`IOException` - If an I/O error occurs.

See Also:

`writeBytes(byte[])`,
`write(byte[],int,int)`

writeBytes

```
public void writeBytes(byte[] buf)
```

Writes all bytes from the specified byte array to this stream. If automatic flushing is enabled then the `flush` method will be invoked.

Note that the bytes will be written as given; to write characters that will be translated according to the default charset, use the `print(char[])` or `println(char[])` methods.

Implementation Requirements:

This method is equivalent to `this.write(buf, 0, buf.length)`.

Parameters:

`buf` - A byte array

Since:

14

print

```
public void print(boolean b)
```

Prints a boolean value. The string produced by `String.valueOf(boolean)` is translated into bytes according to the default charset, and these bytes are written in exactly the manner of the `write(int)` method.

Parameters:

`b` - The `boolean` to be printed

See Also:

`Charset.defaultCharset()`

print

```
public void print(char c)
```

Prints a character. The character is translated into one or more bytes according to the character encoding given to the constructor, or the default charset
if none specified. These bytes are written in exactly the manner of the `write(int)` method.

Parameters:

`c` - The `char` to be printed

See Also:

`Charset.defaultCharset()`

print

```
public void print(int i)
```

Prints an integer. The string produced by `String.valueOf(int)` is translated into bytes according to the default charset, and these bytes are written in exactly the manner of the `write(int)` method.

Parameters:

`i` - The `int` to be printed

See Also:

`Integer.toString(int)`, `Charset.defaultCharset()`

print

```
public void print(long l)
```

Prints a long integer. The string produced by `String.valueOf(long)` is translated into bytes according to the default charset, and these bytes are written in exactly the manner of the `write(int)` method.

Parameters:

`l` - The `long` to be printed

See Also:

`Long.toString(long)`, `Charset.defaultCharset()`

print

```
public void print(float f)
```

Prints a floating-point number. The string produced by `String.valueOf(float)` is translated into bytes according to the default charset, and these bytes are written in exactly the manner of the `write(int)` method.

Parameters:

`f` - The `float` to be printed

See Also:

`Float.toString(float)`, `Charset.defaultCharset()`

print

```
public void print(double d)
```

Prints a double-precision floating-point number. The string produced by `String.valueOf(double)` is translated into bytes according to the default charset, and these bytes are written in exactly the manner of the `write(int)` method.

Parameters:

`d` - The `double` to be printed

See Also:

`Double.toString(double)`, `Charset.defaultCharset()`

print

```
public void print(char[] s)
```

Prints an array of characters. The characters are converted into bytes according to the character encoding given to the constructor, or the default charset
if none specified. These bytes are written in exactly the manner of the `write(int)` method.

Parameters:

`s` - The array of chars to be printed

Throws:

`NullPointerException` - If `s` is `null`

See Also:

`Charset.defaultCharset()`

print

`public void print(String s)`

Prints a string. If the argument is `null` then the string `"null"` is printed. Otherwise, the string's characters are converted into bytes according to the character encoding given to the constructor, or the default charset if none specified. These bytes are written in exactly the manner of the `write(int)` method.

Parameters:

`s` - The `String` to be printed

See Also:

`Charset.defaultCharset()`

print

`public void print(Object obj)`

Prints an object. The string produced by the `String.valueOf(Object)` method is translated into bytes according to the default charset, and these bytes are written in exactly the manner of the `write(int)` method.

Parameters:

`obj` - The `Object` to be printed

See Also:

`Object.toString()`, `Charset.defaultCharset()`

println

`public void println()`

Terminates the current line by writing the line separator string. The line separator string is defined by the system property `line.separator`, and is not necessarily a single newline character (`'\n'`).

println

`public void println(boolean x)`

Prints a boolean and then terminates the line. This method behaves as though it invokes `print(boolean)` and then `println()`.

Parameters:

`x` - The `boolean` to be printed

println

`public void println(char x)`

Prints a character and then terminates the line. This method behaves as though it invokes `print(char)` and then `println()`.

Parameters:

`x` - The `char` to be printed.

println

`public void println(int x)`

Prints an integer and then terminates the line. This method behaves as though it invokes `print(int)` and then `println()`.

Parameters:

`x` - The `int` to be printed.

println

`public void println(long x)`

Prints a long and then terminates the line. This method behaves as though it invokes `print(long)` and then `println()`.

Parameters:

`x` - a The `long` to be printed.

println

`public void println(float x)`

Prints a float and then terminates the line. This method behaves as though it invokes `print(float)` and then `println()`.

Parameters:

`x` - The `float` to be printed.

println

`public void println(double x)`

Prints a double and then terminates the line. This method behaves as though it invokes `print(double)` and then `println()`.

Parameters:

`x` - The `double` to be printed.

println

```
public void println(char[] x)
```

Prints an array of characters and then terminates the line. This method behaves as though it invokes `print(char[])` and then `println()`.

Parameters:

`x` - an array of chars to print.

println

```
public void println(String x)
```

Prints a String and then terminates the line. This method behaves as though it invokes `print(String)` and then `println()`.

Parameters:

`x` - The `String` to be printed.

println

```
public void println(Object x)
```

Prints an Object and then terminates the line. This method calls at first String.valueOf(x) to get the printed object's string value, then behaves as though it invokes `print(String)` and then `println()`.

Parameters:

`x` - The `Object` to be printed.

printf

```
public PrintStream printf(String format,
                          Object... args)
```

A convenience method to write a formatted string to this output stream using the specified format string and arguments.

An invocation of this method of the form `out.printf(format, args)` behaves in exactly the same way as the invocation

```
    out.format(format, args)
```

Parameters:

`format` - A format string as described in Format string syntax

`args` - Arguments referenced by the format specifiers in the format string. If there are more arguments than format specifiers, the extra arguments are

ignored. The number of arguments is variable and may be zero. The maximum number of arguments is limited by the maximum dimension of a Java array as defined by *The Java Virtual Machine Specification*. The behaviour on a `null` argument depends on the conversion.

Returns:

This output stream

Throws:

`IllegalFormatException` - If a format string contains an illegal syntax, a format specifier that is incompatible with the given arguments, insufficient arguments given the format string, or other illegal conditions. For specification of all possible formatting errors, see the Details section of the formatter class specification.

`NullPointerException` - If the `format` is `null`

Since:

1.5

printf

```
public PrintStream printf(Locale l,
                          String format,
                          Object... args)
```

A convenience method to write a formatted string to this output stream using the specified format string and arguments.

An invocation of this method of the form `out.printf(l, format, args)` behaves in exactly the same way as the invocation

```
    out.format(l, format, args)
```

Parameters:

`l` - The locale to apply during formatting. If `l` is `null` then no localization is applied.

`format` - A format string as described in Format string syntax

`args` - Arguments referenced by the format specifiers in the format string. If there are more arguments than format specifiers, the extra arguments are ignored. The number of arguments is variable and may be zero. The maximum number of arguments is limited by the maximum dimension of a Java array as defined by *The Java Virtual Machine Specification*. The behaviour on a `null` argument depends on the conversion.

Returns:

This output stream

Throws:

`IllegalFormatException` - If a format string contains an illegal syntax, a format specifier that is incompatible with the given arguments, insufficient arguments given the format string, or other illegal conditions. For specification of all possible formatting errors, see the Details section of the formatter class specification.

`NullPointerException` - If the `format` is `null`

Since:

1.5

format

```
public PrintStream format(String format,
                          Object... args)
```

Writes a formatted string to this output stream using the specified format string and arguments.

The locale always used is the one returned by `Locale.getDefault(Locale.Category)` with `FORMAT` category specified, regardless of any previous invocations of other formatting methods on this object.

Parameters:

`format` - A format string as described in Format string syntax

`args` - Arguments referenced by the format specifiers in the format string. If there are more arguments than format specifiers, the extra arguments are ignored. The number of arguments is variable and may be zero. The maximum number of arguments is limited by the maximum dimension of a Java array as defined by *The Java Virtual Machine Specification*. The behaviour on a `null` argument depends on the conversion.

Returns:

This output stream

Throws:

`IllegalFormatException` - If a format string contains an illegal syntax, a format specifier that is incompatible with the given arguments, insufficient arguments given the format string, or other illegal conditions. For specification of all possible formatting errors, see the Details section of the formatter class specification.

`NullPointerException` - If the `format` is `null`

Since:

1.5

format

```
public PrintStream format(Locale l,
                          String format,
                          Object... args)
```

Writes a formatted string to this output stream using the specified format string and arguments.

Parameters:

`l` - The locale to apply during formatting. If `l` is `null` then no localization is applied.

`format` - A format string as described in Format string syntax

`args` - Arguments referenced by the format specifiers in the format string. If there are more arguments than format specifiers, the extra arguments are ignored. The number of arguments is variable and may be zero. The maximum number of arguments is limited by the maximum dimension of a Java array as defined by *The Java Virtual Machine Specification*. The behaviour on a `null` argument depends on the conversion.

Returns:

This output stream

Throws:

`IllegalFormatException` - If a format string contains an illegal syntax, a format specifier that is incompatible with the given arguments, insufficient arguments given the format string, or other illegal conditions. For specification of all possible formatting errors, see the Details section of the formatter class specification.

`NullPointerException` - If the `format` is `null`

Since:

1.5

append

```
public PrintStream append(CharSequence csq)
```

Appends the specified character sequence to this output stream.

An invocation of this method of the form `out.append(csq)` when `csq` is not `null`, behaves in exactly the same way as the invocation

```
    out.print(csq.toString())
```

Depending on the specification of `toString` for the character sequence `csq`, the entire sequence may not be appended. For instance, invoking the `toString` method of a character buffer will return a subsequence whose content depends upon the buffer's position and limit.

Specified by:

`append` in interface `Appendable`

Parameters:

`csq` - The character sequence to append. If `csq` is `null`, then the four characters `"null"` are appended to this output stream.

Returns:

This output stream

Since:

1.5

append

```
public PrintStream append(CharSequence csq,
                          int start,
                          int end)
```

Appends a subsequence of the specified character sequence to this output stream.

An invocation of this method of the form `out.append(csq, start, end)` when `csq` is not `null`, behaves in exactly the same way as the invocation

```
    out.print(csq.subSequence(start, end).toString())
```

Specified by:

`append` in interface `Appendable`

Parameters:

`csq` - The character sequence from which a subsequence will be appended. If `csq` is `null`, then characters will be appended as if `csq` contained the four characters `"null"`.

`start` - The index of the first character in the subsequence

`end` - The index of the character following the last character in the subsequence

Returns:

This output stream

Throws:

IndexOutOfBoundsException - If start or end are negative, start is greater than end, or end is greater than csq.length()

Since:

1.5

append

```
public PrintStream append(char c)
```

Appends the specified character to this output stream.

An invocation of this method of the form out.append(c) behaves in exactly the same way as the invocation

```
    out.print(c)
```

Specified by:

append in interface Appendable

Parameters:

c - The 16-bit character to append

Returns:

This output stream

Since:

1.5

charset

```
public Charset charset()
```

Returns the charset used in this PrintStream instance.

Returns:

the charset used in this PrintStream instance

Since:

18

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.